

claude-windows / 7fce5410-4c57-4ec9-a054-be898bda22da

Metadata

```
- Source: `claude-windows`  
- Kind: `claude`  
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\7fce5410-4c57-4ec9-a054-be898bda22da\subagents\workflows\wf_c81dbe7a-7ff\agent-aa8aec16fed070ad2.jsonl`  
- SHA-256: `dad45ff0e4bb03d7be0fe584b6af84c38e5cf8b6474acdb4378b49f485fe36cc`  
- Source modified: `2026-06-25T17:02:29+00:00`  
- Imported at: `2026-07-05T16:48:26+00:00`  
- project: `wf_c81dbe7a-7ff`  
- session_id: `7fce5410-4c57-4ec9-a054-be898bda22da`
```

Transcript

1. user (2026-06-25T17:01:08.865Z)

Adversarially verify this PR-review finding for PR #380 (branch o1-p11/gemini-remote-file-gc is checked out; read the real code with `git diff origin/master...HEAD` + Read the files). Be skeptical ? default to `isReal=false` / `adjustedSeverity=INVALID` if the finding is wrong, already handled by the code, or a non-issue. Do NOT re-run CI.

Finding [LOW] Cloud Run worker (backend.worker) also uploads to Gemini but never sweeps ? the orphan-producing production path is uncovered
Location: backend/worker/__main__.py:13,503 (vs backend/main.py:1345)
Detail: I verified the entrypoint topology. The FastAPI server is COVERED: the only `uvicorn.run(app)` is at backend/main.py:1461 INSIDE `main()`, which runs the sweep at line 1345 first; there is no bare 'backend.main:app' ASGI entrypoint in any deploy script (checked `deploy/cloudrun`, `deploy/gcp`, `pyproject [project.scripts]=backend.main:main`). So the PR's hypothesized 'server path skips it' does NOT apply ? good. The genuine gap is the OTHER entrypoint: `deploy/cloudrun/Dockerfile`'s `ENTRYPOINT` is 'python3 -m backend.worker', a separate module (backend/worker/__main__.py) that composes the SAME pipeline (line 13) and CAN run the Gemini AI handoff unless `--set indexing.skip_ai_handoff=true` (line 10,503). It never calls `_maybe_sweep_gemini_orphans`. So a SIGKILL/OOM of a Cloud Run worker mid-upload ? the exact crash the PR targets ? leaves an orphan that the worker itself will never reclaim. Mitigating context: the worker is an ephemeral one-shot job (pull->run->push->exit), not a long-lived process, so a startup sweep is a less natural fit, and Cloud Run runs are often `skip_ai_handoff=true` (per repo memory). The long-lived server IS the right home and is covered. Flagging as a known scope boundary, not a defect.
Suggested fix: Either accept the scope (server-only sweep) and note it in the PR/docstring, or, if the Cloud Run worker is expected to use Gemini in production, invoke the sweep at the start of the worker's infer path too (or rely on a periodic GC job) so worker-produced orphans are reclaimed.

Return `isReal` + `adjustedSeverity` (INVALID if not real) + one-paragraph reasoning grounded in the actual code.

2. assistant (2026-06-25T17:01:11.298Z)

I'll verify this finding by examining the actual code on the PR branch.

3. user (2026-06-25T17:01:13.044Z)

```
f00b71e feat(01/P11): Gemini remote-file GC ? startup orphan sweep  
86705c1 chore: ruff format gemini_refine.py + trajectory_hybrid.py (#379)  
b72ac85 fix(B2/P17): surface Gemini provider failures in all callers (#378)  
---BRANCH---  
o1-p11/gemini-remote-file-gc
```

4. user (2026-06-25T17:01:13.820Z)

```
backend/main.py | 25 ++++++++
backend/providers/gemini.py | 43 ++++++
tests/test_provider_gemini.py | 104 ++++++
3 files changed, 172 insertions(+)
```

5. assistant (2026-06-25T17:01:16.241Z)

The finding is about the worker entrypoint, not in the diff. Let me read the relevant files.

6. user (2026-06-25T17:01:17.509Z)

```
diff --git a/backend/main.py b/backend/main.py
index 5561cee..ea5a875 100644
--- a/backend/main.py
+++ b/backend/main.py
@@ -64,6 +64,7 @@ from backend.pipeline.segmenters.base import segmenter_registry
 from backend.pipeline.stitcher.compiler import VideoCompiler
 from backend.pipeline.stitcher.watermark_i18n import localize_watermark
 from backend.pipeline.validators.base import registry
+from backend.providers import provider_registry
 from backend.reporting import emit_indexer_report
 from backend.results import Failure # standardized error/result contract
 from backend.utils.ffmpeg import ( # P2a: single ffmpeg/ffprobe resolver
@@ -1184,6 +1185,25 @@ def _enforce_startup_or_exit(config) -> None:
     sys.exit(1)

+def _maybe_sweep_gemini_orphans(config) -> None:
+    """01/P11: best-effort sweep of Gemini File-API uploads orphaned by a previous crashed run.
+    A no-op unless the configured provider is Gemini AND its API key is present. Never raises ?
+    a failed sweep is logged and swallowed so it can't block startup. Extracted for unit
+    tests."""
+    try:
+        provider_name = getattr(config, "ai_provider", "")
+        if provider_name != "gemini":
+            return
+        api_key = os.environ.get("GEMINI_API_KEY")
+        if not api_key:
+            return
+        provider = provider_registry.get("gemini", api_key=api_key, model_name=config.ai_model)
+        deleted = provider.cleanup_orphaned_files()
+        if deleted:
+            logger.info("[STARTUP] Gemini orphan sweep reclaimed %d remote file(s).", deleted)
+    except Exception as e:
+        logger.warning("[STARTUP] Gemini orphan sweep skipped (non-fatal): %s", e)
+
+def main():
+    parser = argparse.ArgumentParser(description="Sports Highlight Indexer Orchestrator")
+    parser.add_argument("--video-path", help="Local path to the MP4 sports video file")
@@ -1319,6 +1339,11 @@ def main():
     if swept:
         logger.warning(f"[JOBS] Swept {swept} stale job(s) from a previous run ? failed.")

+    # 01/P11: sweep Gemini File-API uploads orphaned by a hard process death (the per-call
+    # _safe_delete covers normal/except paths, but a SIGKILL/OOM mid-upload leaves a clip on
+    # Google's servers forever). Best-effort, video-only, grace-gated; never blocks startup.
+    _maybe_sweep_gemini_orphans(global_config)
+
+    # CLI processing mode (no web UI needed). --app/--server always run the server, never CLI
+    single-shot.
+    if args.video_path and not args.server and not args.app:
+        print(f"[CLI] Processing video file: {args.video_path}")
```

7. user (2026-06-25T17:01:18.113Z)

```
__init__.py
__main__.py
__pycache__
```

```
---
```

```
"""Worker dispatcher ? pull from storage ? run the pipeline ? push outputs back.
```

```
python -m backend.worker infer --video-uri <uri> --out-uri <prefix> \
    [--segmenter wasb_hybrid] [--config config.json] [--set k=v ...] [--scratch DIR]
python -m backend.worker train ...          # Gen-0 train-from-bucket (follow-up)
```

The URI scheme selects the storage backend via ``backend.storage`` (a bare path / ``local://`` is an identity passthrough; ``s3://`` / ``gcs://`` is an object store), so existing local runs are byte-for-byte unchanged. Combine with ``RALLY\_DETECTOR\_IMPL=stub`` and ``--set indexing.skip\_ai\_handoff=true`` for a **secret-free, GPU-free** run (no Gemini key) ? the path validated end-to-end on DigitalOcean ([``docs/SETUP\\_NEW\\_MACHINE.md``](#)).

This is a thin orchestrator: it composes the SAME building blocks the ``backend.main`` CLI uses (validator registry ? segmenter ? report), never a forked pipeline.

```
"""
```

```
from __future__ import annotations
```

```
import argparse
import csv
import json
import logging
import os
import re
import sys
import tempfile
from typing import Any, List
```

```
from backend import storage
from backend.compute import ComputeJobSpec, get_compute_backend
from backend.config import (
    StartupCheckError,
    enforce_startup_checks,
    load_config,
    probe_cuda_available,
)
from backend.infrastructure.database import Database
from backend.pipeline.segmenters.base import segmenter_registry
from backend.pipeline.validators.base import registry as validator_registry
from backend.reporting import emit_indexer_report
from backend.results import Failure
from backend.utils.ffmpeg import ffmpeg_bin # P2a: single ffmpeg/ffprobe resolver
from backend.utils.hashing import compute_video_id
```

```
logger = logging.getLogger("backend.worker")
```

```
def _coerce(v: str) -> Any:
    """Coerce a ``--set`` string value to bool/int/float where it round-trips, else str."""
    low = v.lower()
    if low in ("true", "false"):
        return low == "true"
    for cast in (int, float):
        try:
            return cast(v)
        except ValueError:
            pass
    return v
```

```
def _apply_overrides(config: Any, sets: List[str]) -> None:
```

8. assistant (2026-06-25T17:01:20.825Z)

Now let me verify the key claims: (1) the worker can run the Gemini AI handoff, (2) it never calls the sweep, (3) the Dockerfile ENTRYPOINT, and (4) the line references.

9. user (2026-06-25T17:01:22.524Z)

```
10:``--set indexing.skip_ai_handoff=true`` for a **secret-free, GPU-free** run (no Gemini key) ?
39:from backend.reporting import emit_indexer_report
61:    """Apply ``a.b.c=value`` overrides onto the typed config (e.g.
indexing.skip_ai_handoff=true)."""
239:    emit_indexer_report(
503:        help="config override, e.g. --set indexing.skip_ai_handoff=true")
```

10. user (2026-06-25T17:01:23.905Z)

```
deploy/cloudrun/Dockerfile:3:# Runs `python -m backend.worker infer ?` INLINE on an L4 (detect +
window + stitch co-located so
deploy/cloudrun/Dockerfile:69:ENTRYPOINT ["python3", "-m", "backend.worker"]
deploy/cloudrun/README.md:46:python -m backend.worker infer --video-uri
gs://<bucket>/videos/clip.mp4 --out-uri gs://<bucket> --segmenter wasb_hybrid
deploy/gcp/nightly_regression/Dockerfile.launcher:8:ENTRYPOINT ["bash",
"deploy/gcp/nightly_regression/launch.sh"]
deploy/gcp/README.md:168:# in the same shell that runs `python -m backend.worker train ?`.)
deploy/gcp/README.md:208:RALLY_DETECTOR_IMPL=native python -m backend.worker train \
deploy/gcp/README.md:213:python -m backend.worker infer \
---pyproject---
84:[project.scripts]
85:# `indexer --server`, `indexer --help`, etc. -> backend/main.py::main()
86:indexer = "backend.main:main"
```

11. assistant (2026-06-25T17:01:27.165Z)

Confirmed: the worker has no `\_maybe\_sweep\_gemini\_orphans` / `\_cleanup\_orphaned\_files` call, and the Cloud Run Dockerfile ENTRYPOINT is `python3 -m backend.worker`. Now the critical question: does the worker actually run the Gemini AI handoff? Let me examine how the worker composes the pipeline and whether it invokes the AI provider.

12. user (2026-06-25T17:01:28.162Z)

```
180     print(f"[worker] pulled {args.video_uri} -> {local_video}")
181
182     # 2. IDENTITY ? whole-file MD5 (the labels?video join key; pulled bytes must be
byte-identical).
183     video_id = compute_video_id(local_video)
184
185     # 3. VALIDATE (same registry as the main CLI).
186     val_results, val_context = validator_registry.run_all_with_context(local_video,
config)
187     failures = [r for r in val_results if not r.passed]
188     db = Database(os.path.join(scratch, config.output.db_name))
189     db.add_video(video_id, local_video, "failed" if failures else "processed",
190                 [r.model_dump() for r in val_results])
191
192     def _fail(rc: int) -> int:
193         # M6: on a worker-side FAILURE, write the done-marker (the REAL rc, 0 segments)
so a remote
194         # dispatcher's bucket-poll terminates FAST + accurately instead of hanging until
its poll
195         # budget expires. DEFAULT-OFF: no --done-uri ? no marker written (the unchanged
local run).
196         if getattr(args, "done_uri", None):
```

```

197         _write_done_marker(args.done_uri, video_id, rc, 0, "failed", storage_cfg,
scratch)
198         return rc
199
200     segments: List[dict] = []
201     segmenter_actual = None
202     segmentation_ran = False
203     status = "failed"
204     try:
205         if failures:
206             print("[worker] validation FAILED: "
207                   + "; ".join(f"{r.validator_name}: {r.message}" for r in failures))
208             return _fail(2)
209         seg_name = args.segmenter or config.indexing.default_segmenter
210         segmenter_cls = segmenter_registry.get(seg_name)
211         if not segmenter_cls:
212             print(f"[worker] unknown segmenter: {seg_name!r} "
213                   f"(have {list(segmenter_registry.list_available())})")
214             return _fail(2)
215         segmenter_actual = seg_name
216         result = segmenter_cls(db, config).process_video(video_id, local_video,
max_frames=args.max_frames)
217         segmentation_ran = True
218         if isinstance(result, Failure):
219             print(f"[worker] segmenter FAILED: {result.message}")
220             return _fail(3)
221         segments = result.value if hasattr(result, "value") else result
222         status = "success"
223         # Loud, never-silent diagnostic: a 0-segment success is almost always a
misconfig, not an
224         # empty match ? the cold-start failure mode (the substrate detector yielded no
usable
225         # trajectories). Explain it so a run is analysable from the log alone (re-run -v
for more).
226         if not segments:
227             detector_impl = (os.environ.get("RALLY_DETECTOR_IMPL")
228                              or getattr(config.indexing, "detector_impl", None) or
"auto")
229             logger.warning(
230                 "infer produced 0 segments for video_id=%s (segmenter=%s,
detector_impl=%s). "
231                 "The detector substrate yielded no usable trajectories/windows ? check
the detector "
232                 "logs above; common causes: native runner UNAVAILABLE
(weights/repo/WASB_PYTHON env), "
233                 "the WASB env produced no detections, or the input resolution differs
from the tuned "
234                 "proxy resolution. Re-run with -v for the native command + frame
counts.",
235                 video_id, segmenter_actual, detector_impl,
236             )
237         finally:
238             # Best-effort per-video observability report (never raises) ? parity with the
main CLI.
239             emit_indexer_report(
240                 db=db, video_id=video_id, video_path=local_video, config=config,
241                 val_results=val_results, val_context=val_context,
242                 segmenter_requested=args.segmenter, segmenter_actual=segmenter_actual,
243                 was_reingest=False, segmentation_ran=segmentation_ran,
244             )
245
246         # 5. SERIALIZE + 6. PUSH (outputs/<video_id>/?). idempotent on video_id.
247         out_local = os.path.join(scratch, "outputs", video_id)
248         os.makedirs(out_local, exist_ok=True)
249         _write_intervals_csv(segments, os.path.join(out_local, "intervals.csv"))

```

```

250     _write_report_json(video_id, segments, status, os.path.join(out_local,
"report.json"))
251
252     out_prefix = args.out_uri.rstrip("/")
253     pushed = []
254     for fn in sorted(os.listdir(out_local)):
255         uri = f"{out_prefix}/outputs/{video_id}/{fn}"
256         storage.put(os.path.join(out_local, fn), uri, config=storage_cfg)
257         pushed.append(uri)
258
259     print(f"[worker] infer done: {len(segments)} segment(s); pushed {len(pushed)}
artifact(s):")
260     for u in pushed:
261         print("    ", u)
262
263     # M6: a remote dispatcher passes --done-uri; write the SUCCESS completion marker so
its bucket-poll
264     # terminates (carries video_id ? the dispatcher's output path + rc=0). Worker
FAILURES (rc 2/3)
265     # write their own marker via _fail() above, so the dispatcher fails FAST either way
(no 30-min hang
266     # on a bad video). DEFAULT-OFF: no --done-uri ? no marker written (the unchanged
local run).
267     if getattr(args, "done_uri", None):
268         _write_done_marker(args.done_uri, video_id, 0, len(segments), status,
storage_cfg, scratch)
269     return 0
270
271
272     #: Video extensions accepted under <corpus>/videos/ (mirrors the labels/ '.csv' filter
so a
273     #: per-video sidecar ? .srt/.json/thumbnail ? can't shadow the real video on a stem
collision).
274     _VIDEO_EXTS = (".mp4", ".mov", ".mkv", ".avi", ".webm", ".m4v")
275
276     #: Leading ISO-date prefix on a canonical label name (`labels/<YYYY-MM-
DD>_<name>.rallies.csv`,
277     #: docs/DATA_IN_GCS.md §7.1). It disambiguates recycled GoPro names by authoring date
but is NOT
278     #: part of the clip's join key, so it's stripped when deriving the videos/ stem.
279     _LABEL_DATE_PREFIX = re.compile(r"^\d{4}-\d{2}-\d{2}_")
280
281
282     def _label_clip_name(fname: str) -> str:
283         """Derive the clip name (the `videos/<name>.<ext>` join key) from a label
filename.
284
285         Strips both the `.rallies.csv` / `.csv` suffix and an optional leading ISO-date
prefix, so a
286         canonical dated label and a legacy bare label resolve to the SAME bare clip stem:
287
288         * `2026-06-22_GX020094.rallies.csv` -> `GX020094` (canonical, date-
stamped)
289         * `GX020094.rallies.csv` -> `GX020094` (legacy bare ? same
stem)
290         * `GX020094_proxy.rallies.csv` -> `GX020094_proxy` (`_proxy` is part of
the name, kept)
291         * `2026-06-21_mahadevpura_1.rallies.csv` -> `mahadevpura_1` (underscores in
<name> survive)
292
293         Without the date strip, `--trajectory-source detector` can't match a dated label
to its bare
294         video stem, so every clip is skipped and train aborts with `<2 videos`
(DATA_IN_GCS §7b #1).
295         """

```

```

296         name = fname.replace(".rallies.csv", "").replace(".csv", "")
297         return _LABEL_DATE_PREFIX.sub("", name)
298
299

```

13. assistant (2026-06-25T17:01:32.079Z)

This is the crux. The worker's `infer` path runs validate ? segmenter ? serialize ? push. It does NOT call the Gemini AI handoff at all. The AI handoff (`run\_ai\_handoff` / provider calls) is part of `backend.main`'s pipeline, not the worker. Let me confirm the worker never invokes the Gemini provider anywhere, and check where the AI handoff / Gemini upload actually happens.

14. user (2026-06-25T17:01:33.651Z)

```

10:``--set indexing.skip_ai_handoff=true`` for a **secret-free, GPU-free** run (no Gemini key) ?
61:    """"Apply ``a.b.c=value`` overrides onto the typed config (e.g.
indexing.skip_ai_handoff=true).""""
503:                help="config override, e.g. --set indexing.skip_ai_handoff=true")
===END worker grep (empty above = none)===

```

15. user (2026-06-25T17:01:35.226Z)

```

backend/main.py:610:    # skip_ai_handoff=True the WASB windows become rallies directly (no
Gemini key needed); without
backend/main.py:614:    sk = getattr(getattr(cfg, "indexing", None), "skip_ai_handoff", None)
backend/main.py:616:    overrides.append(f"indexing.skip_ai_handoff={'true' if sk else
'false'}")
backend/main.py:1200:    deleted = provider.cleanup_orphaned_files()
backend/providers/gemini.py:44:    def cleanup_orphaned_files(self, grace_sec: int = 3600) ->
int:
backend/providers/gemini.py:46:    crashed run (a hard process death between
``files.upload`` and ``_safe_delete``
backend/providers/gemini.py:116:    video_file =
self.client.files.upload(file=video_path)

```

16. assistant (2026-06-25T17:01:39.050Z)

The worker only mentions `skip\_ai\_handoff` in docstrings/help text ? it never actually reads it or runs an AI handoff. The Gemini upload (`files.upload`) lives in `backend/providers/gemini.py`. The key question: does the worker's segmenter pipeline path ever invoke the Gemini provider? Let me check how the AI handoff is wired ? whether it's inside the segmenter (which the worker DOES call) or only in `backend.main`'s orchestration.

17. user (2026-06-25T17:01:40.708Z)

```

backend/pipeline/segmenters/gemini.py:9:from backend.providers import provider_registry
backend/pipeline/segmenters/gemini.py:93:    provider_name = self.config.ai_provider
backend/pipeline/segmenters/gemini.py:108:    provider =
provider_registry.get(provider_name, api_key=api_key, model_name=model_name)
backend/pipeline/segmenters/gemini.py:222:    chunk_segments, metrics =
provider.analyze_video(
backend/pipeline/segmenters/gemini.py:352:    ai_segments, metrics = provider.analyze_video(
backend/pipeline/segmenters/trajectory_hybrid.py:32:from backend.providers import
provider_registry
backend/pipeline/segmenters/trajectory_hybrid.py:191:    skip_ai =
bool(idx_cfg.skip_ai_handoff)
backend/pipeline/segmenters/trajectory_hybrid.py:192:    provider_name =
self.config.ai_provider
backend/pipeline/segmenters/trajectory_hybrid.py:196:    "%s in FULLY-LOCAL mode
(skip_ai_handoff=True): candidate windows will "
backend/pipeline/segmenters/trajectory_hybrid.py:215:    provider =
provider_registry.get(
backend/pipeline/segmenters/trajectory_hybrid.py:398:    segments, metrics =
provider.analyze_video(

```

```

backend/pipeline/segmenters/yolo_hybrid.py:26:from backend.providers import provider_registry
backend/pipeline/segmenters/yolo_hybrid.py:74:         provider_name = self.config.ai_provider
backend/pipeline/segmenters/yolo_hybrid.py:90:         provider =
provider_registry.get(provider_name, api_key=api_key, model_name=model_name)
backend/pipeline/segmenters/yolo_hybrid.py:285:         segments, metrics =
provider.analyze_video(w_path, prompt, chunk_duration=w_dur, label=f"Handoff {wi+1}")
backend/pipeline/segmenters/yolo_hybrid.py:382:         provider_name = self.config.ai_provider
Binary file backend/pipeline/segmenters/__pycache__/gemini.cpython-313.pyc matches
Binary file backend/pipeline/segmenters/__pycache__/trajectory_hybrid.cpython-313.pyc matches
Binary file backend/pipeline/segmenters/__pycache__/yolo_hybrid.cpython-313.pyc matches

```

18. user (2026-06-25T17:01:43.064Z)

```

backend/eval/gemini_refine.py
backend/main.py
backend/providers/gemini.py
backend/providers/__init__.py

```

19. assistant (2026-06-25T17:01:49.451Z)

This is the decisive finding. The Gemini upload happens INSIDE the segmenters (`trajectory\_hybrid.py`, `yolo\_hybrid.py`, `gemini.py`) via `provider.analyze\_video()`. The worker DOES call `segmenter\_cls(db, config).process\_video(...)` at line 216 ? so when the worker runs `wasb\_hybrid` (which is `trajectory\_hybrid`) without `skip\_ai\_handoff=true`, it WILL invoke `provider.analyze\_video` ? which calls `files.upload`. So the worker genuinely can produce Gemini orphans, and it never calls the sweep. The finding's core technical claim is correct.

Let me verify the `analyze\_video`/upload path and the `\_safe\_delete` mitigation to confirm orphans are actually possible on hard death, and verify the exact line numbers cited.

20. user (2026-06-25T17:01:50.596Z)

```

100         except OSError:
101             size_mb = 0.0
102             if size_mb > MAX_UPLOAD_MB:
103                 logger.error(f"[{log_prefix}] {video_path} is {size_mb:.0f}MB >
{MAX_UPLOAD_MB}MB cap ? "
104                             f"SKIPPED WITHOUT ANALYSIS (no segments for this time range). "
105                             f"Re-encode/downscale chunks (indexing.ai_chunk_scale_width) or
chunk smaller.")
106                 metrics["error"] = f"oversize: {size_mb:.0f}MB > {MAX_UPLOAD_MB}MB cap"
107                 metrics["oversize_skipped"] = True
108                 return [], metrics
109
110         # 1. Upload (retry transient network failures, e.g. WinError 10053 / reset
connections)
111         logger.info(f"[{log_prefix}] Uploading {video_path}...")
112         t_upload_start = time.time()
113         video_file = None
114         for attempt in range(3):
115             try:
116                 video_file = self.client.files.upload(file=video_path)
117                 metrics["upload_time"] = time.time() - t_upload_start
118                 logger.info(f"[{log_prefix}] Upload complete in
{metrics['upload_time']:.1f}s. URI: {video_file.uri}")
119                 break
120             except Exception as e:
121                 logger.warning(f"[{log_prefix}] Upload attempt {attempt + 1}/3 failed:
{e}")
122                 if attempt < 2:
123                     time.sleep(2 * (attempt + 1))
124         if video_file is None:
125             logger.error(f"[{log_prefix}] Upload failed after 3 attempts.")
126             metrics["error"] = "upload failed after 3 attempts"

```

```

127         return [], metrics
128
129     # 2. Wait for processing on Google servers ? a BOUNDED poll (a stuck file must
not hang
130     # the worker forever). Per-poll logs at DEBUG (no INFO spam); see
EXECUTION_POLICY.md.
131     t_process_start = time.time()
132     process_policy = ExecutionPolicy(poll_every_n_sec=10.0, max_wait_sec=1200.0) #
20 min cap
133
134     def _processed():
135         nonlocal video_file
136         video_file = self.client.files.get(name=video_file.name)
137         return None if video_file.state.name == "PROCESSING" else video_file
138
139     try:
140         video_file = poll_until(_processed, process_policy,
141                                label=f"{log_prefix} processing", logger=logger)
142     except PolicyTimeout:
143         logger.error(f"[{log_prefix}] Processing exceeded
{process_policy.max_wait_sec:.0f}s ? giving up.")
144         self._safe_delete(video_file)
145         metrics["error"] = f"processing timed out after
{process_policy.max_wait_sec:.0f}s"
146         return [], metrics
147     except Exception as e:
148         logger.error(f"[{log_prefix}] Status check failed: {e}")
149         self._safe_delete(video_file)
150         metrics["error"] = f"status check failed: {e}"
151         return [], metrics
152     if video_file.state.name == "FAILED":
153         logger.error(f"[{log_prefix}] Video processing failed:
{video_file.error.message}")
154         self._safe_delete(video_file)
155         metrics["error"] = f"video processing failed: {video_file.error.message}"
156         return [], metrics
157
158     metrics["process_time"] = time.time() - t_process_start
159     logger.info(f"[{log_prefix}] Processing finished in
{metrics['process_time']:.1f}s.")

```

21. user (2026-06-25T17:01:51.179Z)

```

188         return []
189
190     idx_cfg = self.config.indexing
191     skip_ai = bool(idx_cfg.skip_ai_handoff)
192     provider_name = self.config.ai_provider
193     if skip_ai:
194         model_name = "local-noai"
195         logger.info(
196             "%s in FULLY-LOCAL mode (skip_ai_handoff=True): candidate windows will "
197             "be emitted as rallies; no %s handoff, no API key needed.",
198             label,
199             provider_name,
200         )
201     else:
202         model_name = self.config.ai_model
203         api_key_env_var = f"{provider_name.upper()}_API_KEY"
204         api_key = os.environ.get(api_key_env_var)
205         if not api_key:
206             from backend.pipeline.segmenters._keyhint import api_key_hint
207
208             logger.error(
209                 f"{api_key_env_var} environment variable is not set! "

```

```

210             f"To run the {provider_name} handoff, set your API key. "
211             + api_key_hint(api_key_env_var)
212         )
213         return []
214     try:
215         provider = provider_registry.get(
216             provider_name, api_key=api_key, model_name=model_name
217         )

```

22. assistant (2026-06-25T17:01:56.290Z)

Confirmed. The mitigation `\_safe\_delete` covers the timeout/exception/failed paths, but a SIGKILL/OOM mid-upload (between `files.upload` at line 116 and the eventual `\_safe\_delete`) leaves an orphan ? exactly the crash the PR's sweep targets. And the worker path runs this segmenter and never sweeps. The finding's technical claims are all accurate.

Now let me assess severity calibration. The finding itself self-rates LOW and frames it as a "known scope boundary, not a defect." Let me verify the mitigating context: is the worker an ephemeral one-shot job, and does Cloud Run typically run with `skip\_ai\_handoff=true`? Let me check the Cloud Run README and the cloudrun-gpu memory note.

23. user (2026-06-25T17:01:57.774Z)

deploy/cloudrun/README.md:53:exit 0 + a 5-frame contact sheet), gotchas, and the **\$0 teardown audit**.

24. user (2026-06-25T17:02:01.139Z)

```

deploy/gcp/nightly_regression/README.md:47:| **Segmenter / detector / no-AI** | `wasb_hybrid` /
`native` / `skip_ai_handoff=true` | `config_overrides` in the manifest. (`skip_ai_handoff=true`
= pure-engine, no paid Gemini ? deterministic + free.) |
deploy/gcp/nightly_regression/README.md:140:| `reels=0`, log mentions GEMINI_API_KEY |
`skip_ai_handoff` not true ? ensure the manifest `config_overrides` has
`indexing.skip_ai_handoff: true` |
artifacts/cloudrun_e2e_validation/halfA_cloudrun_execution.log:27:2026-06-22T16:55:11.787685Z
16:55:11 [INFO] backend.pipeline.segmenters.trajectory_hybrid: WASB in FULLY-LOCAL mode
(skip_ai_handoff=True): candidate windows will be emitted as rallies; no gemini handoff, no API
key needed.
artifacts/cloudrun_e2e_validation/RUN_TRANSCRIPT.md:58:2. **`skip_ai_handoff` gate** ? with
`skip_ai_handoff=False` (the cloud-serving default) and no
artifacts/cloudrun_e2e_validation/RUN_TRANSCRIPT.md:60:  `--set indexing.skip_ai_handoff=true`
(WASB windows ? rallies directly, no key). Both passes used it.
artifacts/cloudrun_e2e_validation/RUN_TRANSCRIPT.md:107:--set,indexing.skip_ai_handoff=true"
artifacts/cloudrun_e2e_validation/RUN_TRANSCRIPT.md:115:  --segmenter wasb_hybrid --set
indexing.skip_ai_handoff=true
docs/BACKLOG.md:14:- ? **One YAML manifest drives the whole bucket?reels run** ? today,
converting `gs://` videos into highlight reels is a sequence of API calls (see
[`CONVERT_GCS_VIDEOS_TO_HIGHLIGHTS.md`](CONVERT_GCS_VIDEOS_TO_HIGHLIGHTS.md)). Propose a
declarative **batch manifest** (`pipeline.yaml`): **inputs** (bucket prefix/glob or explicit
list), **compute** target (local/cloud), **detection** (segmenter, skip_ai_handoff), per-video
**selection** rules (all / top-N longest / min duration / a query string), and **output**
(bucket prefix or local dir + watermark locale) ? driven by one `indexer --batch pipeline.yaml`
runner that loops process?query?select?compile?deliver, **idempotent/resumable** (skips already-
done `video_id`s), reusing the existing API + the compute-picker + the compute-path provenance.
**Why deferred:** needs a small design ? the manifest schema, the selection grammar,
idempotency/resume, and partial-failure handling ? i.e. a tracked project doc per
`PROJECT_DOC_TEMPLATE.md`, not a quick add (owner agreed 2026-06-23: ship the runbook doc now,
backlog the YAML driver). **Revisit:** when hands-off batch conversion is wanted ? start with a
design doc.
docs/CLOUD_GPU_DRYRUN_GUIDE.md:130:  "indexing": { "default_segmenter": "wasb_hybrid",
"detection_impl": "native", "skip_ai_handoff": true },
docs/CLOUD_GPU_DRYRUN_GUIDE.md:139:> **Why `skip_ai_handoff: true`?** The default pipeline does
an optional Gemini "polish" pass that
docs/CLOUD_GPU_DRYRUN_GUIDE.md:193:| `reel.log`: `0 segment(s)` / `No play segments detected` |

```

Almost always a config issue: confirm `config.json` has `"detector\_impl": "native"` **and** `"skip\_ai\_handoff": true`. A non-badminton or very short/blurry clip can also legitimately find no rallies. |

docs/CLOUD_GPU_DRYRUN_GUIDE.md:195:| `reel.log`: `GEMINI\_API\_KEY ... is not set` | You're missing `"skip\_ai\_handoff": true` in `config.json` ? add it (see step 4). |

docs/COMPUTE_DECOUPLED_SERVING/README.md:94:**Exists:** the 3-stage core; `\_resolve\_runner` (the single compute seam); `StubDetectorRunner` (`replay\_csv` = byte-exact \$0 anchor); `StorageBackend` ABC + 4 backends + `StorageRef.parse\_uri`; the worker CLI (fetch/put, `--config`/`--set`); the async job control plane (`jobs` table, `claim\_due\_job` CAS); `ExecutionPolicy WAIT\_AND\_RESUME`/`JobWaiting` (carries to Cloud Run dispatch verbatim); the chunked-upload router (`<2GB`); `skip\_ai\_handoff` (LocalNoAI); storage fakes; golden fixtures + experiment harness. **Partial:** per-video workspace helper (default-OFF); `DeviceContext` (designed); edge-auth (v4 `user\_id` columns exist). **Net-new:** the `deployment` section; `input\_backend`; configurable output base (kill hardcoded `output/`); Cloud Run dispatch shim + container image; cost ledger + pricebook; edge-auth header extraction; startup env detection.

docs/COMPUTE_DECOUPLED_SERVING/README.md:165:- 2026-06-21 ? **REAL M7 cloud GPU run ?** (owner-authorized, within the D17 cap): provisioned an **L4** (`g2-standard-4`, asia-southeast1-a; Mumbai stocked-out) ? native WASB on the L4 ? **22 rallies** on the smoke clip ?

`gs://?/outputs/05e0e577?/{intervals.csv,report.json}`; VM DELETED (\$0, ~\$0.21). ? the no-Gemini cloud run needs `--set indexing.skip\_ai\_handoff=true` (the fully-local path emits WASB candidate windows directly ? the long-flagged "0 rallies" cold-start fix). The reel **stitch** validated locally (`VideoCompiler` ? 22-segment highlights, 85 MB). DEPLOY_REPORT runlog = the cloud-GPU dry-run guide (`docs/CLOUD\_GPU\_DRYRUN\_GUIDE.md`).

docs/CONVERT_GCS_VIDEOS_TO_HIGHLIGHTS.md:75: `indexing`: {"skip_ai_handoff": true},

docs/CONVERT_GCS_VIDEOS_TO_HIGHLIGHTS.md:79: `skip\_ai\_handoff: true` runs the **pure-engine** detection (WASB windows ? rallies, no Gemini key);

docs/NIGHTLY_ENGINE_REGRESSION.md:66: `gs://?/videos/mahadevpura\_smoke\_180s.mp4` (`detector\_impl=native`, `skip\_ai\_handoff=true`):

docs/NIGHTLY_ENGINE_REGRESSION.md:72: `WASB\_WEIGHTS`); weights live at `models/` not `weights/`; `skip\_ai\_handoff=true` for a pure-engine run.

docs/NIGHTLY_ENGINE_REGRESSION.md:91: `skip\_ai\_handoff=true` (pure-engine, no paid Gemini, deterministic + free).

docs/PACKAGING_PLAN.md:15:1. ? **P3 GPU validation ? DONE (2026-06-22, \$0).** Validated the packaged **wheel** end-to-end on a real L4: torch-free wheel install ? `indexer --app` ? bundled SPA ? native-WASB (separate `\$WASB\_PYTHON` conda env) ? **22 rallies ? 73.7 MB watermarked reel**, all via the REAL API. Proof in `gs://khelsutra-rally-corpus/highlights/p3\_{packaging\_highlights.mp4,run.log,app.log}`. VM deleted, \$0 (~\$0.17, ~14 min). The old "0 rallies" blocker was already solved by `skip\_ai\_handoff=true` (`docs/CLOUD\_GPU\_DRYRUN\_GUIDE.md`). Screencast = owner records from the preserved reel/logs.

25. assistant (2026-06-25T17:02:08.693Z)

This fully corroborates the mitigating context. Every documented Cloud Run / GCP worker run uses `skip\_ai\_handoff=true` (the e2e validation log at line 27 literally shows "FULLY-LOCAL mode (skip_ai_handoff=True)... no gemini handoff, no API key needed"). However, the doc at `RUN\_TRANSCRIPT.md:58` explicitly notes `skip\_ai\_handoff=False` is "the cloud-serving default" ? so a production worker run WITHOUT the override flag would do the Gemini handoff and could orphan a file. So the gap is real but only triggers when the worker is run with Gemini enabled, which is not the validated/documented path (always overridden to true).

I have enough to render a verdict. Let me confirm the cited line numbers (13, 503, 1345) are reasonable.

The finding cites `backend/worker/\_\_main\_\_.py:13` (line 13 is in the docstring composing the same pipeline ? accurate), `:503` (the `--set ... skip\_ai\_handoff` help ? accurate), and `backend/main.py:1345` (the sweep call site ? the diff shows it at the line after `1339`, accurate). All citations check out.

I have a complete, grounded picture.

26. user (2026-06-25T17:02:20.147Z)

Structured output provided successfully

27. assistant (2026-06-25T17:02:29.467Z)

Verdict: the finding is REAL at LOW severity.

The finding's technical claims all check out against the actual code:

- ``deploy/cloudrun/Dockerfile:69`` ENTRYPOINT is ``python3 -m backend.worker`` ? a distinct module from ``backend.main``.
- The worker's infer path runs the segmenter (``backend/worker/__main__.py:216``), and the Gemini upload lives inside the hybrid segmenters (``trajectory_hybrid.py`/`yolo_hybrid.py`/`gemini.py`` ? ``provider.analyze_video()`` ? ``client.files.upload()`` at ``backend/providers/gemini.py:116``). So with ``skip_ai_handoff=False`` the worker genuinely uploads to Gemini and can orphan a file on a hard SIGKILL/OOM (the per-call ``_safe_delete`` at lines 144/149/154 only covers timeout/exception/FAILED, not process death).
- The worker never calls ``_maybe_sweep_gemini_orphans``/``cleanup_orphaned_files`` ? it only mentions ``skip_ai_handoff`` in docstrings/help text. So the worker that produces an orphan never reclaims it.

The mitigating context is also confirmed and is what keeps this at LOW: every documented and validated Cloud Run / GCP worker run mandates ``--set indexing.skip_ai_handoff=true`` (the e2e validation log shows "FULLY-LOCAL mode... no gemini handoff, no API key needed"; same in ``CLOUD_GPU_DRYRUN_GUIDE.md``, ``NIGHTLY_ENGINE_REGRESSION.md``, ``RUN_TRANSCRIPT.md``), so in practice the worker uploads nothing to Gemini today. The PR correctly homes the sweep in the long-lived server, which IS covered (the only ``uvicorn.run`` is at ``backend/main.py:1461`` inside ``main()``, after the sweep at line ~1345). This is a legitimate scope-boundary note worth a docstring/PR mention, not a blocking defect ? the finding's own LOW self-rating and "known scope boundary, not a defect" framing are accurate.